

Coding After Coders: The End of Computer Programming as We Know It



In the era of A.I. agents,
many Silicon Valley
programmers are now
barely programming.
Instead, what they're doing
is deeply, deeply weird.



Illustration by Pablo Delcan and Danielle Del Plato

By Clive Thompson

Clive Thompson interviewed more than 70 software developers at Google, Amazon, Microsoft and small start-ups. He is the author of “Coders: The Making of a New Tribe and the Remaking of the World.”

March 12, 2026

Lately, Manu Ebert has been trying to keep his A.I. from humiliating him.

I recently visited Ebert, a machine-learning engineer and former neuroscientist, at the spare apartment where he and Conor Brennan-Burke run their start-up, Hyperspell. Ebert, a tall and short-bearded 39-year-old with the air of a European academic, sat before a mammoth curved monitor.

Onscreen, Claude Code — the A.I. tool from Anthropic — was busy at work. One of its agents was writing a new feature and another was testing it; a third supervised everything, like a virtual taskmaster. After a few minutes, Claude flashed: “Implementation complete!”

Ebert grew up in the '90s, learning to code the old-fashioned way: He typed it out, line by painstaking line. After college, he held jobs as a software developer in Silicon Valley for companies like Airbnb before becoming a co-founder of four start-ups. Back then, developing software meant spending days hunched over his keyboard, pondering gnarly details, trying to avoid mistakes.

All that ended last fall. A.I. had become so good at writing code that Ebert, initially cautious, began letting it do more and more. Now Claude Code does the bulk of it. The agents are so fast — and generally so accurate — that when a customer recently needed Hyperspell to write some new code, it took only half an hour. In the before times? “That alone would have taken me a day,” he said.

He and Brennan-Burke, who is 32, are still software developers, but like most of their peers now, they only rarely write code. Instead, they spend their days talking to the A.I., describing in plain English what they want from it and responding to the A.I.'s “plan” for what it will do. Then they turn the agents loose.

A.I. being A.I., things occasionally go haywire. Sometimes when Claude misbehaves and fails to test the code, Ebert scolds the agent: Claude, you really *do* have to run all the tests.

To avoid repeating these sorts of errors, Ebert has added some stern warnings to his prompt file, the list of instructions — a stern Ten Commandments — that his agents must follow before they do anything. When you behold the prompt

file of a coder using A.I., you are viewing a record of the developer's attempts to restrain the agents' generally competent, but unpredictably deviant, actions.

I looked at Ebert's prompt file. It included a prompt telling the agents that any new code had to pass every single test before it got pushed into Hyperspell's real-world product. One such test for Python code, called a pytest, had its own specific prompt that caught my eye: "Pushing code that fails pytest is unacceptable and embarrassing."

Embarrassing? Did that actually help, I wondered, telling the A.I. not to "embarrass" you? Ebert grinned sheepishly. He couldn't prove it, but prompts like that seem to have slightly improved Claude's performance.

His experience is not unusual; many software developers these days berate their A.I. agents, plead with them, shout important commands in uppercase — or repeat the same command multiple times, like a hypnotist — and discover that the A.I. now seems to be slightly more obedient. Such melodramatic prose might seem kind of nuts, but as their name implies, large language models are language machines. "Embarrassing" probably imparted a sense of urgency.

"If you say, *This is a national security imperative, you need to write this test*, there is a sense of just raising the stakes," Ebert said.

Brennan-Burke chimed in: "You remember seeing the research that showed the more rude you were to models, the better they performed?" They chuckled. Computer programming has been through many changes in its 80-year history. But this may be the strangest one yet: It is now becoming a conversation, a back-and-forth talk fest between software developers and their bots.

This vertiginous shift threatens to stir up some huge economic consequences. For decades, coding was considered such wizardry that if you were halfway competent you could expect to enjoy lifetime employment. If you were exceptional at it (and lucky), you got rich. Silicon Valley panjandrums spent the 2010s lecturing American workers in dying industries that they needed to “learn to code.”

Now coding itself is being automated. To outsiders, what programmers are facing can seem richly deserved, and even funny: American white-collar workers have long fretted that Silicon Valley might one day use A.I. to automate *their* jobs, but look who got hit first! Indeed, coding is perhaps the first form of very expensive industrialized human labor that A.I. can actually replace. A.I.-generated videos look janky, artificial photos surreal; law briefs can be riddled with career-ending howlers. But A.I.-generated code? If it passes its tests and works, it’s worth as much as what humans get paid \$200,000 or more a year to compose.

You might imagine this would unsettle and demoralize programmers. Some of them, certainly. But I spoke to scores of developers this past fall and winter, and most were weirdly jazzed about their new powers.

“We’re talking 10 to 20 — to even 100 — times as productive as I’ve ever been in my career,” Steve Yegge, a veteran coder who built his own tool for running swarms of coding agents, told me. “It’s like we’ve been *walking* our whole lives,” he says, but now they have been given a ride, “and it’s fast as [expletive].” Like many of his peers, though, Yegge can’t quite figure out what it means for the future of his profession. For decades, being a software developer meant mastering coding languages, but now a language technology itself is upending the very nature of the job.

The enthusiasm of software developers for generative A.I. stands in stark contrast to how other Americans feel about the impact of large language models. Polls show a majority are neutral or skeptical; creatives are often enraged. But if coders are more upbeat, it's because their encounters with A.I. are diametrically opposite to what's happening in many other occupations, says Anil Dash, a friend of mine who is a longtime programmer and tech executive. "The reason that tech generally — and coders in particular — see L.L.M.s differently than everyone else is that in the creative disciplines, L.L.M.s take away the most soulful human parts of the work and leave the drudgery to you," Dash says. "And in coding, L.L.M.s take away the drudgery and leave the human, soulful parts to you."



Operators working with an IBM 7090 computer in 1960, at what was then known as the Lawrence Radiation Laboratory in Livermore, Calif. Lawrence Livermore National Laboratory/Science Photo Library

Coding *has* been drudgery, historically. In the movies, programmers excitedly crank out code at typing speed. In reality, writing software has always been an agonizingly slow and frustrating affair. You write a few lines of code, a single “function” that does one little thing, and then discover that you made some niggling error, like leaving out a single colon. As a company’s “codebase” — every line of code in its software, accreting over the years — gets larger and involves dozens or thousands of functions interacting with one another, you could spend hours, days or weeks pulling your hair out trying to find which little mistakes are bringing everything to a halt. Maybe a line of yours broke something your colleague is coding two cubicles over.

For decades, computer engineers tried to automate this drudgery. In the industry, they call every step in this direction “adding a layer of abstraction”: If you often find yourself doing something step by step in an onerous fashion, you automate it.

For example, one early computer language was Assembly, and it was devilishly hard to write. Computers had very little memory, so coders had to be efficient in how they used it, putting each bit of data carefully in place and then keeping mental track of it. Even simple calculations required an incremental, meticulous approach. Say you wanted to write some code that would calculate how much you’d have if you got 5 percent interest on \$10,000 over 10 years. Back in the 1960s, that would have required perhaps nine lines of pretty obtuse Assembly: “VAL, FLDECML 10000.0” to set the starting amount at \$10,000, “CLA VAL” to load the amount into the processor, “FAD ZERO” to tell the computer you’re working with numbers that have decimal points; and so on.

By the ’80s and ’90s, as computers became more powerful, engineers were able to create languages that took care of all that memory management for you, and also turned common asks into simple commands. In Python, a coder can perform that exact same calculation very simply: “total_amount = 10000 *

(1.05 ** 10).” That single line tells the computer to multiply 10,000 by the interest rate over 10 years and store the result in the variable labeled “total_amount.” Programmers no longer need to think about where all the data is being stored in the computer’s memory; Python does that for them. It is, in other words, a layer of abstraction on top of all that fiddly memory business. Writing in that language is delightfully easier.

An early IBM personal computer and printer introduced in 1981. By the '80s and '90s, as computers became more powerful, engineers created languages that simplified the coding process. IBM/Science Photo Library

During the 2000s and 2010s, programmers abstracted away more and more scut work. Virtually anytime they encountered an onerous task, they wrote some code to automate it and then — very often — made it open source, giving it away for others to use. Here’s an example: As a hobbyist programmer, I

sometimes want to automatically “scrape” the text from a website. I’ve never written code myself to do that; I just use Beautiful Soup, a freely available package of thousands of lines of Python code that manages all the complexity. I don’t even need to understand *how* Beautiful Soup works. It just gives me simple, typically one-line Python commands that — whoosh — retrieve and analyze website text for me. A significant amount of software is produced in precisely this way: developers stitching together big piles of code that someone else wrote.

With A.I., though, programmers ascend to an even higher level of abstraction. They describe, in regular language, what the program should do, and the agents translate that idea — that human *intent* — into code. Writing software no longer means mentally juggling the nuances of a language like Python, say, or JavaScript or Rust. Coding no longer involves messing up an algorithm and then trying to figure out where your error lies. That part, too, has been abstracted away.

So what exactly is left? Or as Boris Cherny, the head of Claude Code, put it when we met at Anthropic’s headquarters in January: “What is computation — what is coding?” Then he added, “You can get pretty philosophical pretty fast.”

His answer echoed what I’ve heard from pretty much every developer I’ve spoken to: A coder is now more like an architect than a construction worker. Developers using A.I. focus on the overall shape of the software, how its features and facets work together. Because the agents can produce functioning code so quickly, their human overseers can experiment, trying things out to see what works and discarding what doesn’t. Several programmers told me they felt a bit like Steve Jobs, who famously had his staffers churn out prototypes so he could handle lots of them and settle on what felt right. The work of a developer is now more judging than creating.

Cherny himself has been through all the layers of abstraction: As a teenager in California, he taught himself a little Assembly so he could write a program that solved math homework automatically on his calculator. Today he simply pulls out his phone and dictates to Claude what he wants the A.I. agent to do; in a sort of Ouroboric loop, 100 percent of Cherny's contributions to the Claude codebase are now written entirely by Claude.

While we talked, his phone was sitting on the table in front of us, and at the end of an hour he showed me the screen: 10 Claude agents had been tweaking the codebase. "I haven't written a single line by hand, and I'm like the most prolific coder on the team," he said. "It's an alien intelligence that we're learning to work with."

For most of the coders I met, learning to work with A.I. means learning to *talk* to A.I. This struck me as an unexpected paradox of this new age, because traditionally coding was a haven for introverts who preferred to talk as little as possible to others at work. But now their entire job involves constantly chatting with this alien life form.

If describing and talking are now much of the work of a software developer, the talk nonetheless remains pretty complex and highly technical. An amateur can't do it. You can't just tell an agent, *Build me the code for a successful start-up*. The agents work best when they're being asked to perform one step at a time; ask for too much and they can lose the plot. Aayush Naik, whose start-up in San Francisco uses Claude Code, says it's a delusion to imagine that your A.I. agent will generate a whole project at once, in a "Big Bang" moment. Yes, you can get it to write 5,000 lines of code — but then, he says, "you test it and nothing works." This, all the software developers say, is where their training and expertise are still needed: knowing how a big codebase ought to be structured, how to design the system so it's reliable and how to figure out if the agent is sloppy.

Given A.I.'s penchant to hallucinate, it might seem reckless to let agents push code out into the real world. But software developers point out that coding has a unique quality: They can tether their A.I.s to reality, because they can demand the agents test the code to see if it runs correctly. "I feel like programmers have it easy," says Simon Willison, a tech entrepreneur and an influential blogger about how to code using A.I. "If you're a lawyer, you're screwed, right?" There's no way to automatically check a legal brief written by A.I. for hallucinations — other than face total humiliation in court.

When I visited Dima Yanovsky at his small San Francisco apartment, he, too, was busily chatting with Claude. He's a quick-to-smile 25-year-old programmer at Prox, a company that uses A.I. to help e-commerce companies. He founded it last year with his childhood friend Gregory Makodzeba. Both of them grew up in Ukraine, where their families were in the shipping business.

As he dictated commands to Claude, a number of agents were busy at work on his laptop, which was perched on his small desk. At one point, one of them started hallucinating, insisting that a table of data existed that clearly didn't exist. "What?" Yanovsky said, peering at his screen with a frown. He mashed out a disdainful reprimand on his keyboard: "who told you there is gonna be this table? i havent created this table."

Claude replied, in a daft and chipper tone: "You're right! I shouldn't assume tables exist." It began to redo the work.

Even with this occasional backtracking, Claude codes so much faster than Yanovsky that he struggles to put a number on how much faster he can now get his work done. "Like, 20X?" he offered. What once took weeks now takes hours. Every Silicon Valley founder he knows is experiencing the same thing. If you want to build a company in a hurry, nobody does it by hand anymore.

The fact that A.I. can boost coder productivity so drastically has been one of the more remarkable talking points in the field. I've noticed this myself: Just last week, I needed a web tool to clean up some messy transcripts, and I used A.I. to build it in about 10 minutes. On my own, it would have taken an hour, possibly longer.

But software start-ups — or individuals like me who are vibe-coding their own small apps — are a special case. They involve what's known in the industry as “greenfield” coding, where there are no pre-existing lines of code to deal with. An entirely new codebase is being created from scratch.

A vast majority of software developers aren't working in greenfield contexts. They're “brownfield,” employed by mature companies, where the code was written years (or decades) earlier and already reaches millions or billions of lines. *Rapidly* adding new functions is usually a terrible idea — they might accidentally conflict with another part of the code and break something that millions of customers rely on. At most mature software firms, coders historically spent a minority of their time — sometimes barely more than an hour per day — actually writing code. The rest was planning, hashing out priorities and meeting to discuss progress.

This is the curse of success, and why big, established software firms can be slower to deliver upgrades than younger companies. Before a coder's new work is released, colleagues and higher-ups typically put it through a “code review,” looking carefully at its lines and the results of any testing. If you want to put a number on how much more productive A.I. is making the programmers at mature tech firms like Google, it's 10 percent, Sundar Pichai, Google's chief executive, has said.

That's the bump that Google has seen in “engineering velocity” — how much faster its more than 100,000 software developers are able to work. And that 10 percent is the average inside the company, Ryan Salva, a senior director of

product at the company, told me. Some work, like writing a simple test, is now tens of times faster. Major changes are slower. At the start-ups whose founders I spoke to, closer to 100 percent of their code is being written by A.I., but at Google it is not quite 50 percent.

I visited Salva in Sunnyvale, Calif., to shoulder-surf as he showed me how L.L.M.s have been woven into Google's work flow. For a firm with billions of lines of code, he noted, A.I.'s value isn't necessarily in writing new code so much as in figuring out what's going on with the existing lines. Developers will use it to analyze and explain what "sprawling" portions of code are doing, so they can determine how to help improve or alter it.

"A.I. is much better at wading into an unfamiliar part of the codebase, making sense of what's happening," he told me. It also helps developers work in languages they might not be very familiar with. As a result, developers on Salva's team form smaller groups: A year ago, these might have needed 30 people, each with their own specialty. Now a group needs only three to six people, which enables them to move more nimbly, so "we're able to clear through a lot more of our backlog," Salva said.

Salva opened up his code editor — essentially a word processor for writing code — to show me what it's like to work alongside Gemini, Google's L.L.M. For the first few years of the A.I. boom, he said, it was still "very much what I would describe as 'human in the loop.'" The A.I. assisted but didn't work independently. While he typed away, Gemini analyzed a piece of code for him, explaining whether it had been fully tested or not. When it suggested a few new lines, it was up to him to accept them.

But Google's metabolism is gradually speeding up, and Gemini is writing much more code on its own. Salva showed me an example. He had been hankering to solve a problem that Google coders had been complaining about:

Sometimes they would log into Gemini's "command line interface" (or C.L.I.) from different accounts, and it was not easy to see which account they were using.

He typed out a request for Gemini: "When working inside of Gemini C.L.I., it would be nice to have a command that lets users see their logged-in identity." The A.I. processed the request for a few minutes, then told Salva how it intended to fulfill it. Salva gave his approval, and Gemini worked away in the background. When he checked back in 10 minutes, the code had been written and Gemini was testing it for errors. Then Salva realized the A.I. had become a bit overeager.

"Oh, Jesus," he said. "It ran 8,000 tests," far more than was strictly necessary. About 15 minutes later, though, the tests were finished, and Salva tried the new function. Lo and behold, the code worked, correctly displaying his logged-in account. "Not bad," he said. Making a demo like this was only the first baby step; before it could be incorporated into Google's codebase, it would have to go through several rounds of code review, rewriting and testing.

"As an engineer, I care less that the models are really good at producing the right result the first time," he said. "I care much more that there are validation steps in place so that it eventually gets the perfect or the right answer."

A 10 percent increase in Google's "velocity" may seem underwhelming, Salva noted, given the hoopla around A.I. "We have collectively — both in the software industry as well as in the media — oh, my God, created a hype cycle," he had told me when we first talked, last summer in New York. But the reality was impressive enough for him. "We should be delighted when there's 10 percent efficiency gains for the entire company. That's freaking bonkers!"

At old and huge brownfield companies, where the effort is focused on keeping the existing systems up and running, many programmers work like digital plumbers, fixing leaks that erupt at all hours. I saw that firsthand when I met

in Seattle with David Yanacek, a senior principal engineer for AWS Agentic A.I. “AWS” stands for “Amazon Web Services,” the server cloud that is the digital backbone for millions of firms. If a server crashes, you might not be able to watch Netflix, hail an Uber or play Fortnite.

An old-school pager sat beneath Yanacek’s monitor. For years, Amazon used it to wake him during middle-of-the-night incidents; these days, he gets a smartphone alert. Whatever the devices involved, someone is expected to fix things as soon as possible.

“Server ops is *annoying*,” said Yanacek, a trim man of 42 with a gray beard and jittery intensity. “I actually love it! But it’s also annoying, and it’s nonstop.” His team has long built automations to speed up the pace of diagnosing problems. But L.L.M.s have offered powerful new ones, he said, because the A.I.’s fluency in both human language and programming means it can interpret error reports from crashed systems and look at their code. It can sometimes have a fix ready even before a bleary-eyed employee is fully awake.

Yanacek looked at his screen and noticed that, 11 minutes earlier, a demo application had issued an error alert — and Amazon’s A.I. had already pinpointed what went wrong and written a short report. The agent had discovered that a code change had apparently added a new time-stamp field, but some other part of the codebase wasn’t expecting that new field to be there. The result was an “unexpected field” error.

Yanacek peered at the A.I.’s suggested fix, pondered for a moment, then hit “enter” to approve it.

The A.I. took about eight minutes to figure things out, he told me. “By the time I’d opened my laptop, it’s ready.” One customer recently told him that Amazon’s A.I. agent fixed a problem in only 15 minutes; when a similar

problem occurred months before, it had taken a full team of engineers eight hours to debug.

In other Amazon sectors, the brownfield engineers work on revising segments of old code (sometimes decades old) to make them more efficient, or perhaps to redo them entirely in a more modern language. It's work that is crucial but finicky and delicate, like performing a heart transplant.

These digital renovations have sped up, too. McLaren Stanley, a senior principal engineer at Amazon, recently modernized a piece of code he had personally written years earlier. The original version had taken a month to create; this time, with the help of Amazon's in-house A.I., he finished the job in a morning. His team has similarly reworked other big chunks of code. One of A.I.'s key advantages, Stanley told me, is that it makes it easier to try out new ideas. "Things I've always wanted to do now only take a six-minute conversation and a 'Go do that,'" he says.

Grace Hopper, a computer-science pioneer and proponent of creating easier-to-use computer languages, codes problems onto punch tape. Bettmann/Getty Images

I've written about developers for decades, and they have always rhapsodized about the thrill of bringing a machine to life through arcane commands. Sure, the work could be cosmically exasperating, requiring hours or even weeks to chase down a single bug. But the grind sharpened the joy. When things finally started working, the burst of satisfaction was intoxicating.

So I was surprised by how many software developers told me they were happy to no longer write code by hand. Most said they still feel the jolt of success, even with A.I. writing the lines. “I love programming. I love getting in the zone. I love thinking big thoughts. It’s the creative act,” says Kent Beck, a longtime guru of the software industry who has been coding since 1972. Ten years ago, he mostly stopped writing software; he was frustrated with the latest languages and software tools. But L.L.M.s got him going again, and he’s now cranking out more projects than ever: a personalized note-taking app, new types of databases. Even the fact that A.I.’s output can be unpredictable — if you ask it to write a piece of code, it might do so in a slightly different way each time — “is addictive, in a slot-machine way.”

A few programmers did say that they lamented the demise of hand-crafting their work. “I believe that it can be fun and fulfilling and engaging, and having the computer do it for you strips you of that,” one Apple engineer told me. (He asked to remain unnamed so he wouldn’t get in trouble for criticizing Apple’s embrace of A.I.) He went on: “I didn’t do it to make a lot of money and to excel in the career ladder. I did it because it’s my passion. I don’t want to outsource that passion.” He also worries that A.I. is atomizing the work force. In the past,

if developers were stuck on an intractable bug, they asked colleagues for advice; today they just ask the agents. But only a few people at Apple openly share his dimmer views, he said.

The coders who still actively avoid A.I. may be in the minority, but their opposition is intense. Some dislike how much energy it takes to train and deploy the models, and others object to how they were trained by tech firms pillaging copyrighted works. There is suspicion that the sheer speed of A.I.'s output means firms will wind up with mountains of flabbily written code that won't perform well. The tech bosses might use agents as a cudgel: *Don't get uppity at work — we could replace you with a bot.* And critics think it is a terrible idea for developers to become reliant on A.I. produced by a small coterie of tech giants.

Thomas Ptacek, a Chicago-based developer and a co-founder of the tech firm Fly.io, has seen the lacerating fights between the developers who love A.I. and those few who hate it, and "it's a civil war," he told me. He's in the middle. He thinks the refuseniks are deluding themselves when they claim that A.I. doesn't work well and that it *can't* work well. "It's like being gaslit," he says. The holdouts are in the minority, and "you can watch the five stages of grief playing out."

He's not a Pollyanna, though. "L.L.M.s are going to win on coding, but I don't know what that's going to mean for us," he adds. "People may be right about how bad that is for the profession, right?"

It certainly could mean terrible job prospects. New computer-science graduates are particularly concerned. Companies used to hire junior developers to do the menial labor for their senior colleagues, but who is going to hire a neophyte when a senior engineer can be even more productive with an army of deathless code-writing ghosts?

Silicon Valley has already been through a huge wave of layoffs. During the 2010s, tech firms were hiring aggressively, competing for new grads and adding an average of 74,000 new employees a year, according to the Bureau of Labor Statistics. Job postings soared in the early years of the pandemic. Then firms abruptly reversed course, and postings for new jobs collapsed. More than 700,000 tech workers have been laid off in the last four years, according to Roger Lee at Layoffs.fyi (this number includes all jobs in tech).

Most tech observers say A.I. probably wasn't the cause of those layoffs because, at the time, it wasn't yet good enough to replace coders. Other factors, they figure, were more significant: Interest rates rose, so tech firms lost their easy growth money. Companies that overhired shed that excess capacity. Some also suspect that when Elon Musk bought Twitter and said he laid off 80 percent of his work force, tech executives at other firms took note and decided that maybe they didn't need so many engineers either.

But there's evidence that A.I. is now eroding entry-level coding jobs. Last year, Erik Brynjolfsson, an economist who directs the Stanford Digital Economy Lab, and his colleagues analyzed industries based on their age group and how easily their jobs could be done by A.I. He found that computer programmers had one of the most "A.I.-exposed" jobs — and junior developers were hit the hardest. The number of jobs for those between the ages 22 and 25 (when one is most likely to be entering the field) had declined by 16 percent since 2022, while older programmers saw no significant decrease.

Virtually all of the tech executives I've spoken to, from those at coastal giants to those at small regional firms, have sworn to me that A.I. would not stop them from hiring appealing new talent. It's true that A.I. makes their existing developers more productive, but they always need more done.

“In my many years at Google, we have always been constrained by having way, way, way more ideas of things we would like to do than there was time and energy and hours in the day to go do them,” Jen Fitzpatrick, the company’s senior vice president for Google Core Systems & Experiences, told me. “I have never met a team at Google who says, ‘You know, I’m out of good ideas.’ The answer is always, ‘The list of things I would like to do is nine miles longer than what we can pull off.’”

Several developers suggested, in fact, that the number of software jobs might actually grow. An untold number of small firms around the country would love to have their own custom-made software, but were never big enough to hire, say, a five-person programmer team necessary to produce it. But if you could hire a *single* A.I.-assisted coder to do that same work, or even a part-time one? This is, as Brynjolfsson notes, a version of the “Jevons paradox”: When something gets cheaper to do, we don’t just pocket the savings — we do more of it. Though it could also be that these software jobs won’t pay as well as in the past, because, of course, the jobs aren’t as hard as they used to be. Acquiring the skills isn’t as challenging.

This question of skills can lead in some unsettling directions, though, when you chase it down. Many midcareer coders told me they felt confident using A.I. because they had spent decades developing a strong sense of what good, efficient code looks like. That allows them to explain to the agents precisely what they want and lets them spot quickly when the agents have cranked out something inefficient or sloppy.

But what happens to the next generation? Will they still develop that intuitive sense for code? If your job is now less about writing than assessing, how will newbies learn to assess?

Some new developers told me they can feel their skills weakening. Pia Torain is a software engineer for Point Health A.I., and she was only two years into her job when, in the summer of 2024, the company told her to start using Github's Copilot code-writing tool. "I realized that it was just four months that I was prompting hundreds, 500 prompts a day, that I started to lose my ability to code," she says. She stopped using them for a while; these days, she'll have A.I. write for her, but she carefully reads the output, making sure she's absorbing how the code works. "If you don't use it," Torain told me, "you're going to lose it."

Point Health co-founder Rachel Gollub is less worried. She has been a software developer for almost 40 years, and for decades coders have worried that the craft is imminently doomed. When languages like Python and JavaScript emerged, they abstracted away the need to think about memory management, so developers stopped needing those skills. The old-school coders caterwauled: *It's not real coding unless you're managing your own memory!*

"People were all like, 'You're losing all your ability to code,'" Gollub told me. But plenty of big, reliable companies — Dropbox, say — relied heavily on newer languages like Python, and they have worked fine. Memory management is crucial in only a subset of coding tasks today, such as with devices that don't have much computing power. The vast majority of the software industry has moved on. Gollub expects the same transition will happen as A.I. tools become the norm.

Writing code is now so highly abstracted that nearly anyone *could* crack open a L.L.M. and describe an app. Maybe not a complex one. But if they needed some simple software for personal use? An A.I. could likely craft it.

This is what Maxime Cuisy recently did. He's a production manager for a print shop in Paris that produces photo books for high-end clients including Dior and Louis Vuitton. Educationally, he's your classic liberal-arts grad, having completed a master's thesis on the French graphic novel. He knows nothing of coding, and didn't even pay much attention to A.I. until a couple of years ago, when he says ChatGPT "basically helped me and my wife to save our cat."

They had gotten two new kittens, and both became so sick that one suddenly died. The vet told them the remaining cat had terminal cancer. Cuisy thought that was improbable, so he explained the cat's symptoms to ChatGPT, which suggested it was an infection. This inspired him to do more research and led him to a diagnosis of feline infectious peritonitis. A day later, the cat was on the mend.

At work, Cuisy soon had a different problem. The company had bought new printers only to run into problems with their existing software: To get the photos to display correctly, they now had to painstakingly adjust the margins. The company isn't big enough to have a developer team that could make custom software to automate this for them. Cuisy decided to try vibe-coding the solution himself, using Codex, OpenAI's code-writing tool.

"I basically told it, 'I need to have an app that does this, and this is the form factor that the printer can receive,'" he says. He spent a few hours carefully detailing the way files would need to be adjusted, and by the end of the day ChatGPT had produced an app that works on Mac and Windows operating systems. Employees use it to process up to 2,000 images in a single shot. His boss is happy. Cuisy has no idea how the code actually works. It's written in Python, which might as well be ancient Greek.

This is the cultural side effect of coding becoming conversational: The realms of programmers and everyday people, separated for decades by an ocean of arcane know-how, are drifting closer together. If code-writing A.I. continues to

improve, there will likely be far more people in Cuisy's situation — the Jevons paradox in action. "Maybe they don't label themselves as software engineers, but they're creating code," Brynjolfsson says. "A lot of people have ideas." The world becomes flooded with far more software than ever before — written by individuals, for individuals.

How things will shake out for professional coders themselves isn't yet clear. But their mix of exhilaration and anxiety may be a preview for workers in other fields. Anywhere a job involves language and information, this new combination of skills — part rhetoric, part systems thinking, part skepticism about a bot's output — may become the fabric of white-collar work. Skills that seemed the most technical and forbidding can turn out to be the ones most easily automated. Social and imaginative ones come to the fore. We will produce fewer first drafts and do more judging, while perhaps feeling uneasy about how well we can still judge. Abstraction may be coming for us all.

Read by James Patrick Cronin Narration produced by Tanya Pérez Engineered by Devin Murphy

A correction was made on March 13, 2026: An earlier version of this article included a passage with inconsistent examples of coding languages. The passage was meant to demonstrate how to program a function that would calculate how much money would accrue with interest after 10 years, not just the amount of interest accrued.

When we learn of a mistake, we acknowledge it with a correction. If you spot an error, please let us know at corrections@nytimes.com. [Learn more](#)